

cv_eda_hurricane_damage

October 24, 2025

```
[1]: import json
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

1 Loading the data

To get started let's first load in the data! We will make use of the helper functions provided in `data_utils.py`

```
[2]: from data_utils import get_images, get_labels
```

The following is the list of disasters in our dataset.

```
[3]: disaster_list = ["hurricane-matthew", "socal-fire", "midwest-flooding"]
```

We will load each disaster train dataset into a dictionary. Recall that each disaster consists of images and labels (0 - 3) of the damage level.

```
[4]: data = {}
split = "train"
with open('config.json') as config_file:
    config = json.load(config_file)
    data_dir = config['data_dir']

for disaster in disaster_list:
    print(f"Loading {split} images and labels for {disaster} dataset...")
    images = get_images(data_dir, disaster, split=split)
    labels = get_labels(data_dir, disaster, split=split)
    data[disaster] = {"images": images, "labels": labels}
```

Loading train images and labels for hurricane-matthew dataset...
Loading train images and labels for socal-fire dataset...
Loading train images and labels for midwest-flooding dataset...

Let's visualize an image and its label in the dataset.

```
[5]: disaster = disaster_list[0]
images = data[disaster]["images"]
labels = data[disaster]["labels"]

# index of the image to display
idx = 0
img = images[idx]
label = labels[idx]

plt.title(f"{disaster}, image {idx}, label {label}")
plt.imshow(img.astype(np.uint8))
plt.axis("off")
plt.show()

#
print(f"image shape: {img.shape}")
```

hurricane-matthew, image 0, label 1



image shape: (1024, 1024, 3)

2 Extracting Color Information

Each image is an RGB image represented as a numpy array of dimensions (width, height, 3). The last dimension corresponds to the RGB *color channels* in the image. We can split the image by color channels and visualize each of them individually.

```
[6]: plt.figure(figsize=(12, 12))

plt.subplot(2, 2, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(img[:, :, 0], cmap="Reds", vmin=0, vmax=255)
plt.title("Red Channel")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(img[:, :, 1], cmap="Greens", vmin=0, vmax=255)
plt.title("Green Channel")
plt.axis("off")

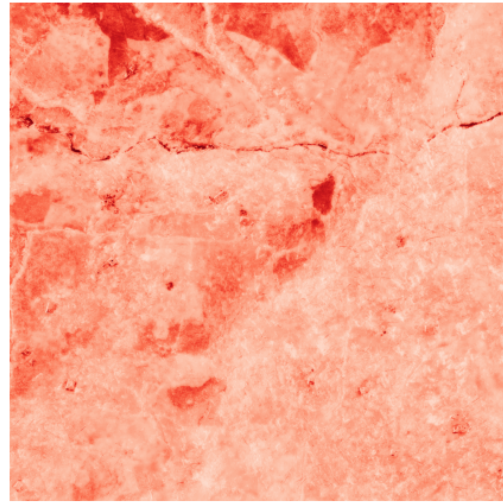
plt.subplot(2, 2, 4)
plt.imshow(img[:, :, 2], cmap="Blues", vmin=0, vmax=255)
plt.title("Blue Channel")
plt.axis("off")

plt.show()
```

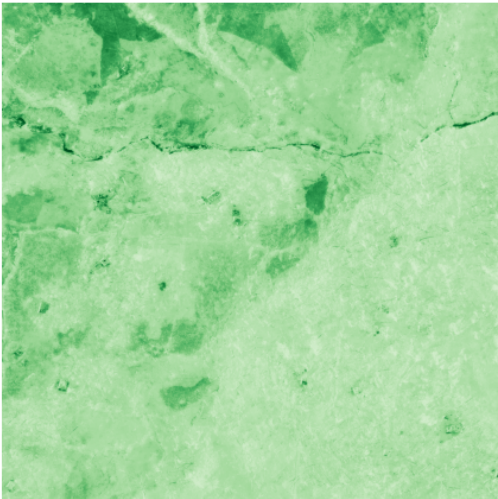
Original Image



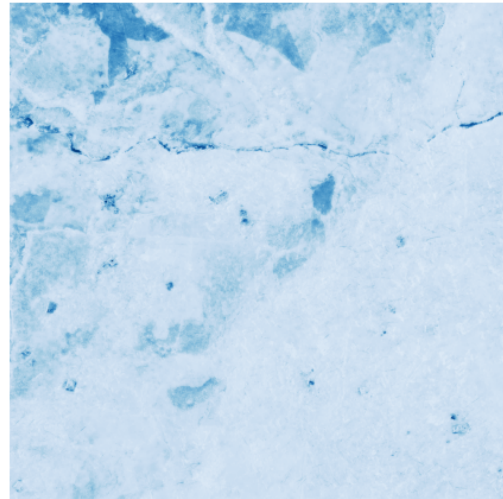
Red Channel



Green Channel



Blue Channel

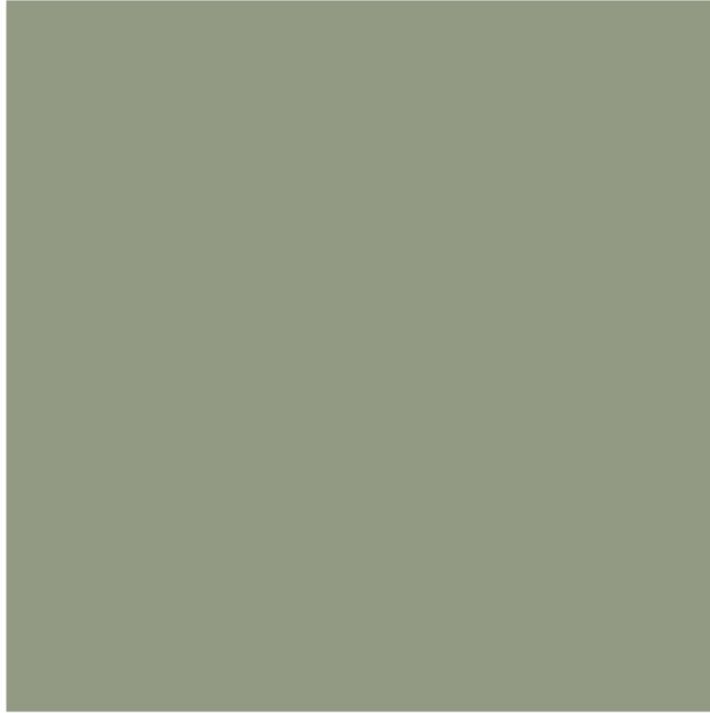


We can zoom in on just a pixel as well.

```
[7]: coord = (15, 35)
    pixel_value = img[coord[0], coord[1], :]
    pixel_image = np.array([[pixel_value]])

    plt.imshow(pixel_image)
    plt.axis("off")
    plt.show()

    print(f"Pixel value: {pixel_value}, coordinate: {coord}")
```

Pixel value: [146 154 131], coordinate: (15, 35)

3 Extracting Edge and Texture Information

So far we have considered the **color** information in an image. However, other important aspects of an image are the edges and texture.

Let us consider some popular feature processing methods for extracting edge and texture information. We will use the utilities provided in `feature_utils.py`.

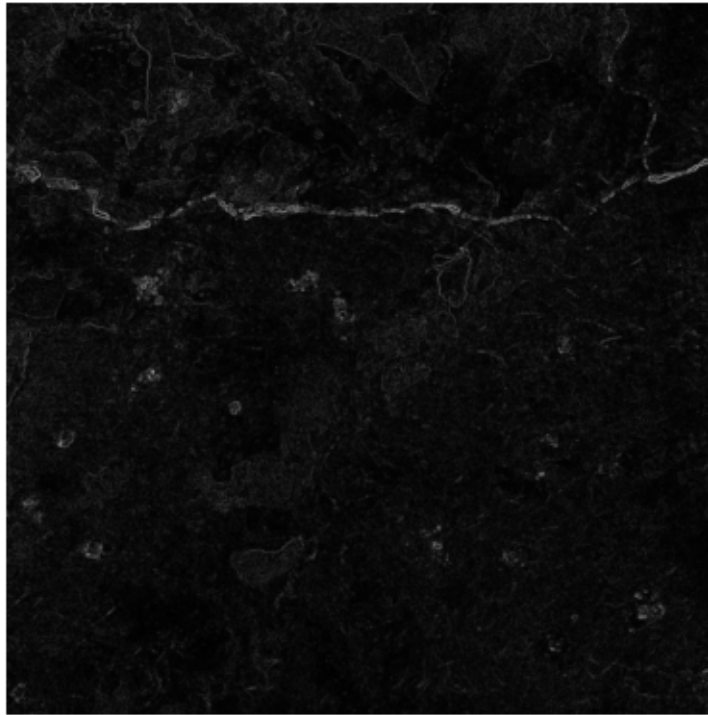
```
[8]: from feature_utils import get_sobel_features, get_gabor_features,   
      ↪ generate_gabor_kernel, get_local_binary_pattern
```

Let's try the Sobel Edge Filter.

Read more about Sobel edge detection: https://en.wikipedia.org/wiki/Sobel_operator and https://scikit-image.org/docs/stable/auto_examples/edges/plot_edge_filter.html

```
[9]: edges = get_sobel_features(img)   
     plt.imshow(edges, cmap="gray")   
     plt.title("Sobel Edge Detection")   
     plt.axis("off")   
     plt.show()
```

Sobel Edge Detection

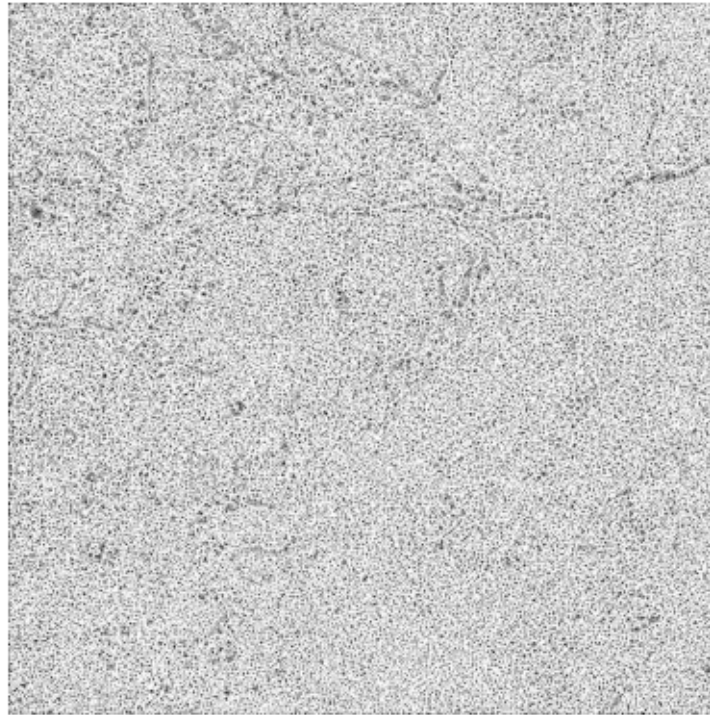


Now let's look at local binary patterns.

Read more about LBP: https://en.wikipedia.org/wiki/Local_binary_patterns and https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_local_binary_pattern.html

```
[10]: lbp = get_local_binary_pattern(img, radius=3)
plt.imshow(lbp, cmap="gray")
plt.axis("off")
plt.title("Local Binary Pattern (LBP)")
plt.show()
```

Local Binary Pattern (LBP)



Another popular filter for image processing is the Gabor filter. The Gabor filters are a family of filters parametrized by orientation, frequency, bandwidth, etc. Let's generate one such filter.

Read more about Gabor filters: https://en.wikipedia.org/wiki/Gabor_filter and https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_gabor.html

```
[11]: theta = 0
      sigma = 1.0
      frequency = 0.1

      kernel = generate_gabor_kernel(theta, sigma, frequency)
      gabor = get_gabor_features(img, kernel)
```

Let's visualize this filter (aka kernel) and the response of the image when we apply the filter. Concretely we are taking the convolution of the image with the filter. See [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing)) for more details.

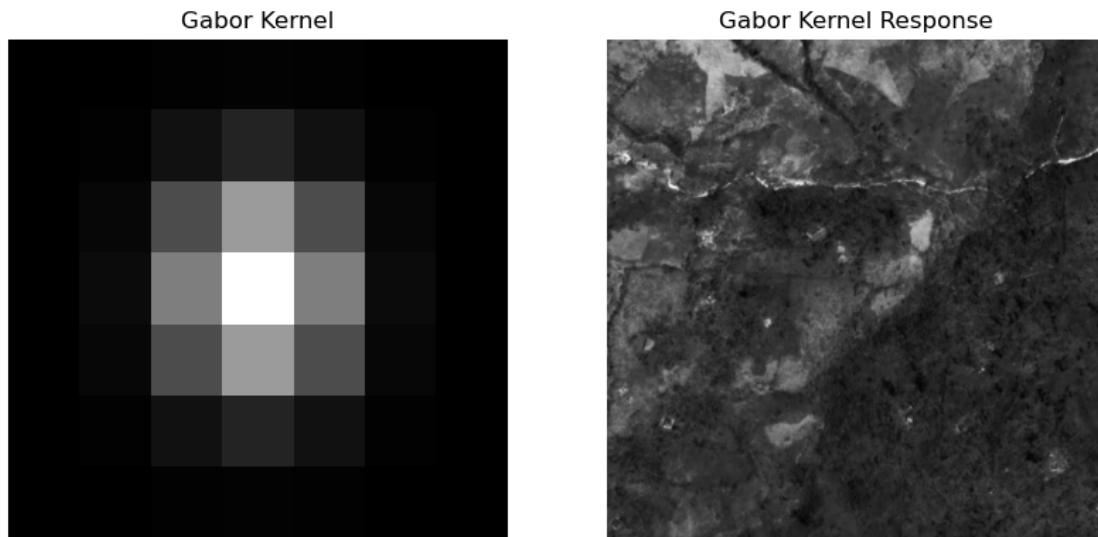
```
[12]: fig, axs = plt.subplots(1, 2, figsize=(10, 5)) # 1 row, 2 columns

      # Plot Gabor Kernel
      axs[0].imshow(kernel, cmap="gray")
      axs[0].axis("off") # Remove axis
      axs[0].set_title("Gabor Kernel")
```

```
# Plot Gabor Kernel Response
axs[1].imshow(gabor, cmap="gray")
axs[1].axis("off") # Remove axis

axs[1].set_title("Gabor Kernel Response")

plt.show()
```



4 Exploratory Data Analysis

The agency would like you to start performing some exploratory data analysis to build an initial understanding of the data. As part of the exploratory data analysis, the agency is interested in understanding certain aspects of the dataset better.

Feel free to add more cells as you see fit for your analysis.

5 EDA - Basic Statistics

5.1 Number of Images per Disaster Type

```
[13]: for disaster in disaster_list:
        print(f"There are {len(data[disaster]['images'])} images for disaster type_{disaster}")
```

There are 200 images for disaster type hurricane-matthew

There are 200 images for disaster type socal-fire

There are 200 images for disaster type midwest-flooding

5.2 Distribution of Image Sizes in Each Dataset

```
[14]: # Print the images sizes for each disaster type
for disaster in disaster_list:
    print(f"Image Sizes for diaster type {disaster}")
    img_x_sizes = np.asarray([im.shape[0] for im in data[disaster]['images']])
    img_y_sizes = np.asarray([im.shape[1] for im in data[disaster]['images']])
    img_num_pixels = img_x_sizes*img_y_sizes
    print(f"X dimension: {img_x_sizes[0]} pixels, same for every image in_
↪dataset: {np.all(img_x_sizes == img_x_sizes[0])}")
    print(f"Y dimension: {img_y_sizes[0]} pixels, same for every image in_
↪dataset: {np.all(img_y_sizes == img_y_sizes[0])}")
    print(f"Total number of pixels: {img_num_pixels[0]} pixels, same for every_
↪image in dataset: {np.all(img_num_pixels == img_num_pixels[0])}")
    print("\n")
```

Image Sizes for diaster type hurricane-matthew

X dimension: 1024 pixels, same for every image in dataset: True

Y dimension: 1024 pixels, same for every image in dataset: True

Total number of pixels: 1048576 pixels, same for every image in dataset: True

Image Sizes for diaster type socal-fire

X dimension: 1024 pixels, same for every image in dataset: True

Y dimension: 1024 pixels, same for every image in dataset: True

Total number of pixels: 1048576 pixels, same for every image in dataset: True

Image Sizes for diaster type midwest-flooding

X dimension: 1024 pixels, same for every image in dataset: True

Y dimension: 1024 pixels, same for every image in dataset: True

Total number of pixels: 1048576 pixels, same for every image in dataset: True

5.3 Distribution of Damage Labels

Let's investigate the frequency of each damage level for the different disaster types, i.e. how many times each damage level occurs in the dataset.

```
[15]: # Plot the distribution of damage labels for each disaster type
for disaster in disaster_list:
    # Define a figure
    fig = plt.figure(figsize=(6, 4))

    # Count how many times each label occurs
    counts = np.bincount(data[disaster]['labels'])
    # Define label values (0-3)
```

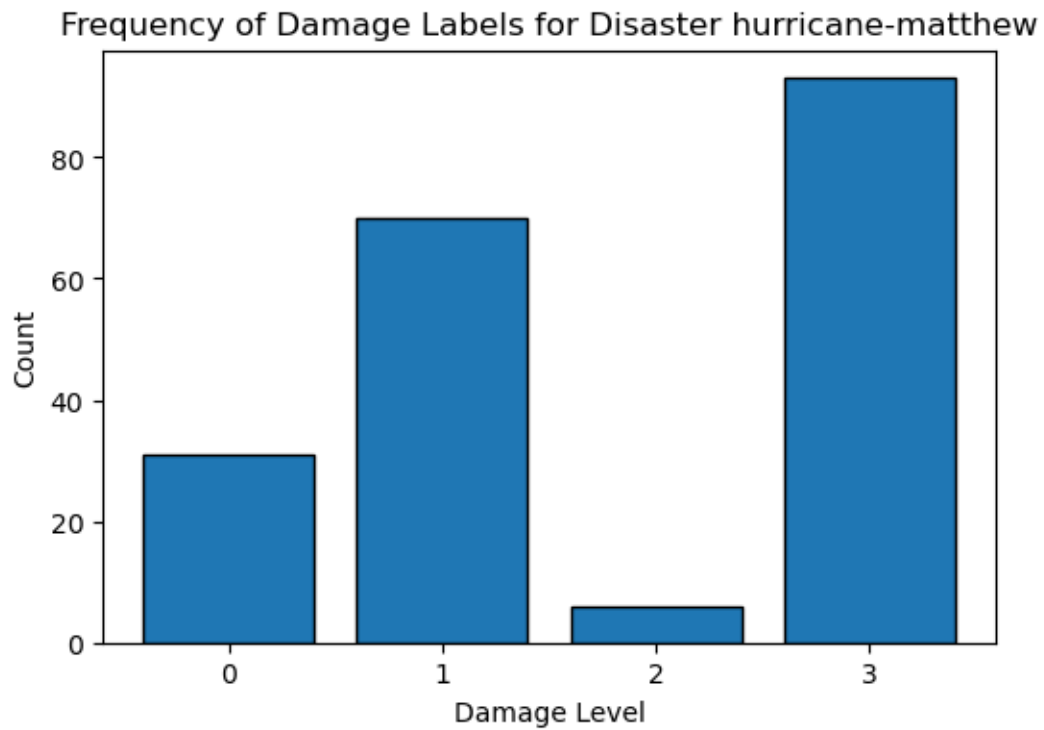
```

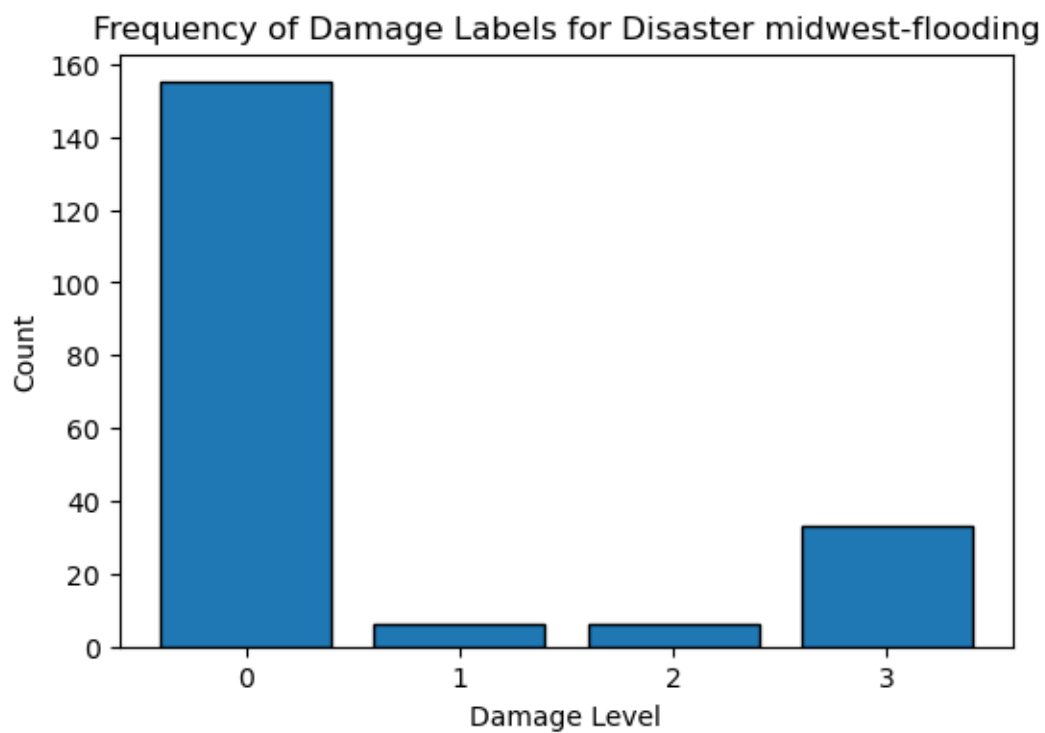
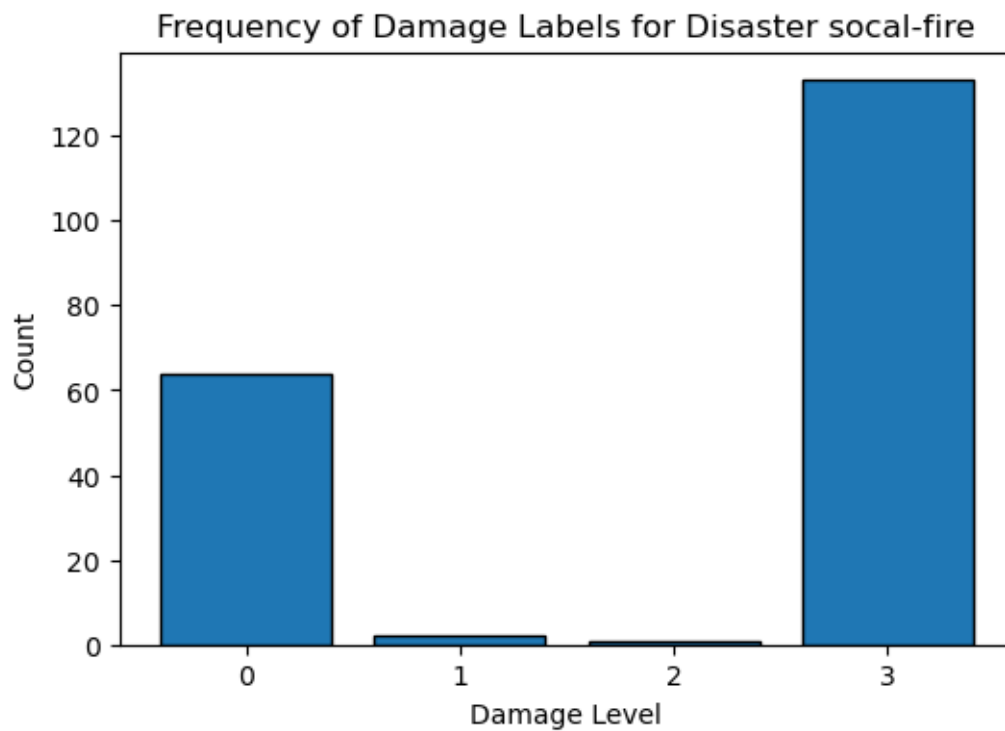
x = np.arange(len(counts))

# Plot bar graph
plt.bar(x, counts, edgecolor='black')
plt.xlabel('Damage Level')
plt.ylabel('Count')
plt.title('Frequency of Damage Labels for Disaster ' + disaster)
plt.xticks(x)
plt.show()

fig.tight_layout()
save_path = './eda_images/damage_label_dist_' + disaster + '.png'
fig.savefig(save_path)

```





6 EDA - Useful Image Features for Classification

The agency is looking for insights into useful image features for classifying images based on disaster type or damage level. You are tasked with verifying this and exploring whether color, edge, or texture features can effectively differentiate:

1. midwest-flooding from socal-fire images.
2. Damage levels 1 and 3 within the hurricane-matthew dataset.

Previous interns have found color information to be potentially useful.

Let's structure the data in such a way that is conducive to studying hurricane damage level 1 vs damage level 3, in this case both flattened and nominal shaped arrays of images for each class.

```
[16]: # Separation of hurricane images into damage level classes

# Gets list of nominal shaped images and flattened images
himgs_dl1, himgs_dl3, himgs_flt_dl1, himgs_flt_dl3 = [], [], [], []
for i, img in enumerate(data['hurricane-matthew']['images']):
    if data['hurricane-matthew']['labels'][i] == 1:
        himgs_dl1.append(img)
        himgs_flt_dl1.append(img.reshape(-1, 3))
    elif data['hurricane-matthew']['labels'][i] == 3:
        himgs_dl3.append(img)
        himgs_flt_dl3.append(img.reshape(-1, 3))

# Fully flatten the flat arrays - all of the pixels (still separated by RGB)
↳ for both classes
himgs_flt_dl1 = np.asarray(himgs_flt_dl1).reshape(-1, 3)
himgs_flt_dl3 = np.asarray(himgs_flt_dl3).reshape(-1, 3)

# Print shapes of these arrays for the nominal shaped images
print('Nominal Images:')
print(f'Shape of damage level 1 images: {np.shape(himgs_dl1)}')
print(f'Shape of damage level 3 images: {np.shape(himgs_dl3)}')

# Print shapes of these arrays for the flattened images
print('Flattened Images:')
print(f'Shape of damage level 1 images: {np.shape(himgs_flt_dl1)}')
print(f'Shape of damage level 3 images: {np.shape(himgs_flt_dl3)}')
```

Nominal Images:

Shape of damage level 1 images: (70, 1024, 1024, 3)

Shape of damage level 3 images: (93, 1024, 1024, 3)

Flattened Images:

Shape of damage level 1 images: (73400320, 3)

Shape of damage level 3 images: (97517568, 3)

```
[17]: # Let's visualize the first 20 images of the damage level 1 dataset
print('Damage Level 1 Images')
for i, img in enumerate(himgs_dl1):
    if i < 20:
        fig, ax = plt.subplots()
        ax.imshow(img)
        ax.set_axis_off()
```

Damage Level 1 Images





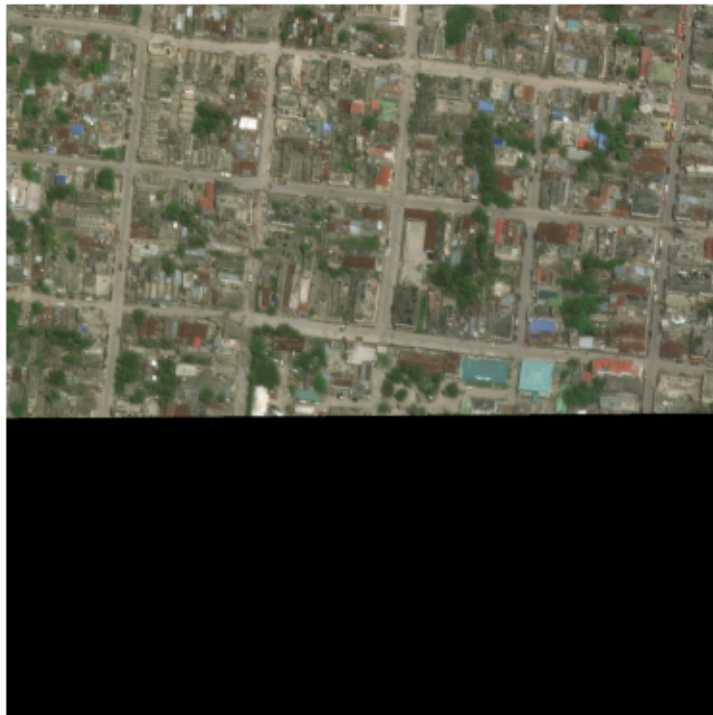












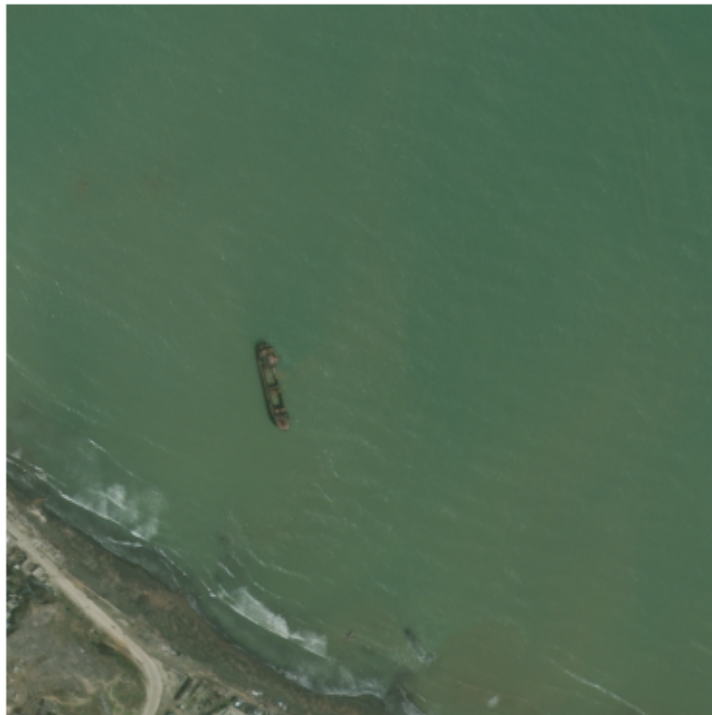






```
[18]: # Let's visualize the first 20 images of the damage level 3 dataset
print('Damage Level 3 Images')
for i, img in enumerate(himgs_dl3):
    if i < 20:
        fig, ax = plt.subplots()
        ax.imshow(img)
        ax.set_axis_off()
```

Damage Level 3 Images





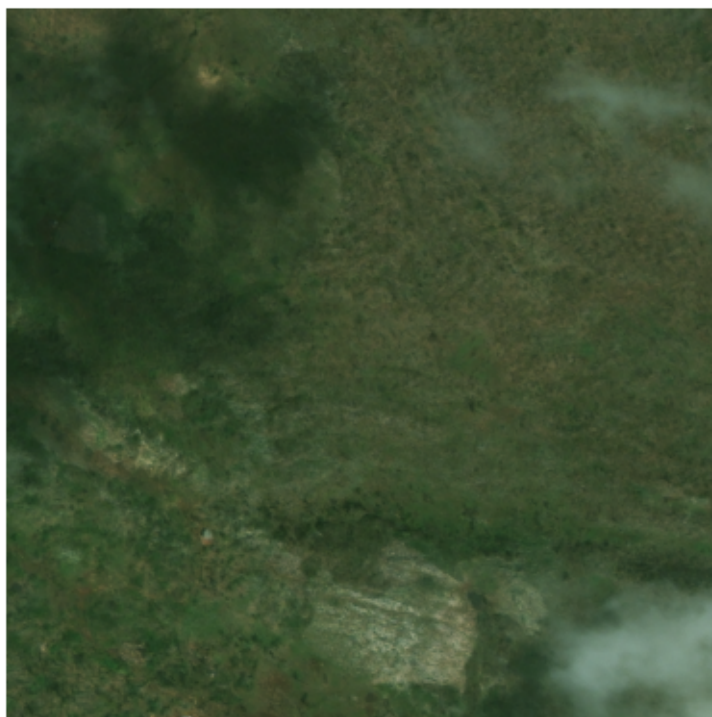
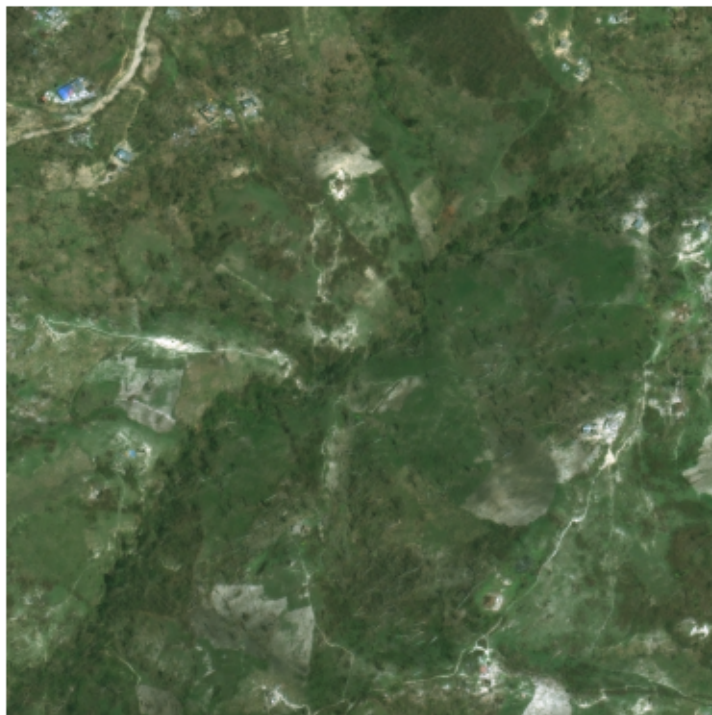
















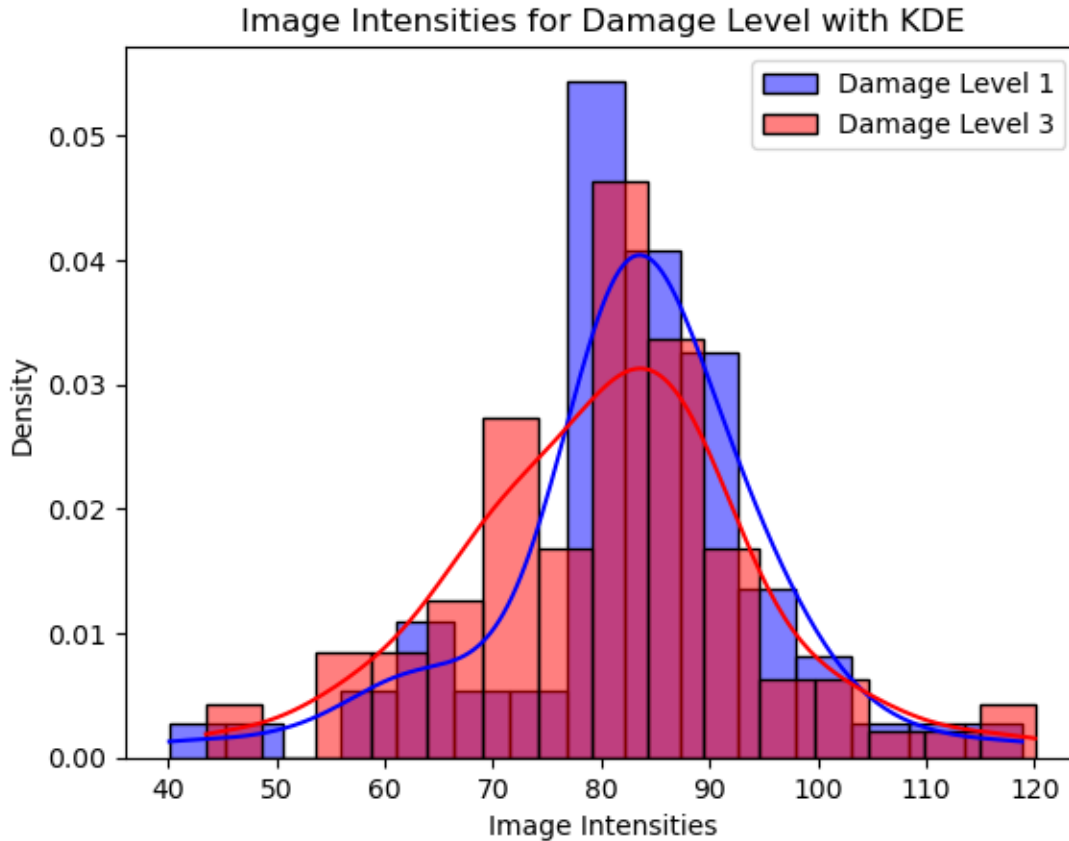
6.1 Distribution of Color (RGB)

Let's look at the distribution of color for hurricane images of damage level 1 and 3. First, we calculate the mean color in each color component (RGB) per image to look at the overall intensity.

```
[19]: # Get the means in the red, green and blue for each image, for each damage level
rgb_means_dl1 = np.vstack([np.mean(np.mean(dl1_img, axis=0), axis=0) for
    ↪ dl1_img in himgs_dl1])
rgb_means_dl3 = np.vstack([np.mean(np.mean(dl3_img, axis=0), axis=0) for
    ↪ dl3_img in himgs_dl3])

# Convert these means RGB values into intensity (mean of the RGB vector)
intensity_dl1 = np.mean(rgb_means_dl1, axis=1)
intensity_dl3 = np.mean(rgb_means_dl3, axis=1)

[20]: # Plot intensity as a normalized histogram distribution for each population
fig = plt.figure()
sns.histplot(intensity_dl1, bins=15, kde=True, stat='density', color='blue',
    ↪ label='Damage Level 1', alpha=0.5)
sns.histplot(intensity_dl3, bins=15, kde=True, stat='density', color='red',
    ↪ label='Damage Level 3', alpha=0.5)
plt.xlabel('Image Intensities')
plt.ylabel('Density')
plt.title('Image Intensities for Damage Level with KDE')
plt.legend()
plt.show()
plt.tight_layout()
fig.savefig('./eda_images/color_intensities.png')
```



<Figure size 640x480 with 0 Axes>

We don't see much difference in the profile of intensities between the classes. Next, let's break it down by color, and look at the overall distributions of color in the images for each damage class. Note that these distributions use all pixels in all images for each class.

```
[21]: # Plot the overall distribution in intensities in RGB channels for damage
      ↪ levels 1 and 3
      colors = ['red', 'green', 'blue']
      fig = plt.figure(figsize=(12, 4))

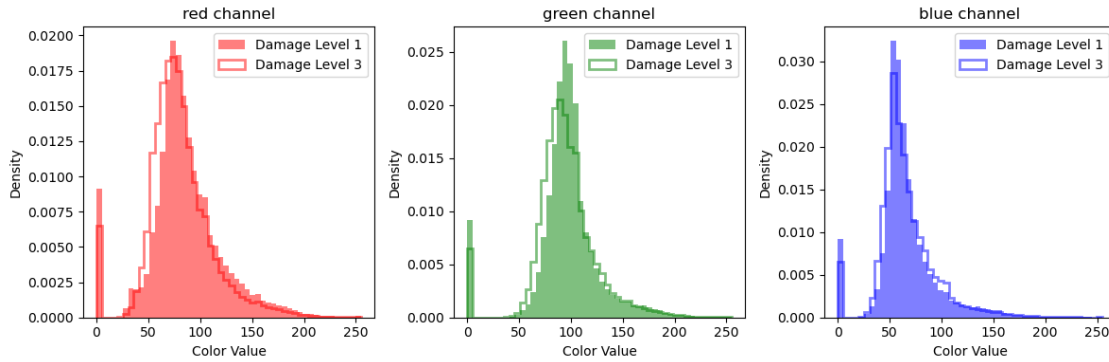
      # For each of the colors, red green and blue
      for i, c in enumerate(colors):
          plt.subplot(1, 3, i + 1)
          plt.hist(himgs_flt_dl1[:, i], bins=50, alpha=0.5, label='Damage Level 1',
                  ↪ color=c, density=True)
          plt.hist(himgs_flt_dl3[:, i], bins=50, alpha=0.5, label='Damage Level 3',
                  ↪ color=c, histtype='step', linewidth=2, density=True)
          plt.title(f'{c} channel')
          plt.xlabel('Color Value')
```

```

plt.ylabel('Density')
plt.legend()

plt.tight_layout()
plt.show()
fig.savefig('./eda_images/rgb_color_distribution.png')

```



The differences in the color profiles between damage level 1 and 3 are fairly minor, which means the populations are not separable based on color distribution in the images.

As a final test for color, let's look at the various permutations of red green and blue colors, to see if there are any particular color combinations which allow us to separate the classes. We create scatter plot projections of each permutation of the red / green / blue planes. This is performed over all pixels of each class, but downsampled to show only every 500th pixel to avoid overcrowding.

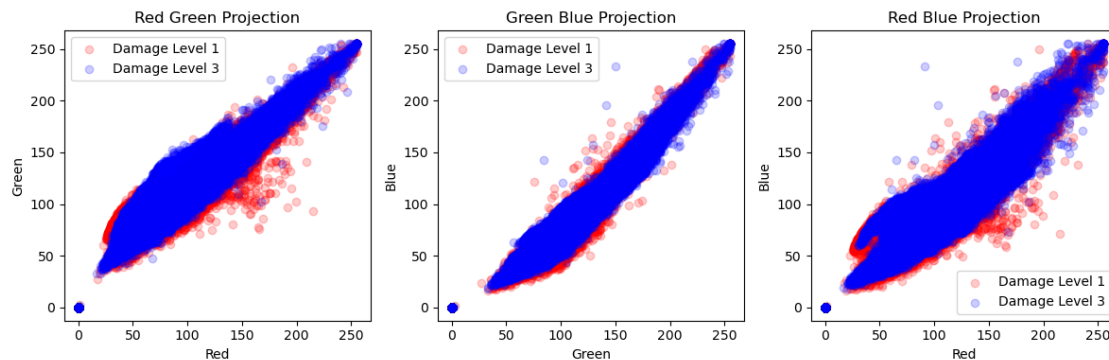
```

[22]: # Define 3 subfigures in which the projections are plotted
fig = plt.figure(figsize=(12, 4))
pairs = [(0,1), (1,2), (0,2)]
labels = [('Red','Green', 'Red Green Projection'), ('Green','Blue', 'Green Blue_
↪Projection'), ('Red','Blue', 'Red Blue Projection')]

# Plot each permutation
for i, ((x, y), (lx, ly, lt)) in enumerate(zip(pairs, labels)):
    plt.subplot(1, 3, i+1)
    # Note that we use every 500th point to avoid overcrowding
    plt.scatter(himgs_flt_dl1[:,500], x, himgs_flt_dl1[:,500], y, alpha=0.2,
↪label='Damage Level 1', color='r')
    plt.scatter(himgs_flt_dl3[:,500], x, himgs_flt_dl3[:,500], y, alpha=0.2,
↪label='Damage Level 3', color='b')
    plt.xlabel(lx)
    plt.ylabel(ly)
    plt.title(lt)
    plt.legend()
plt.tight_layout()

```

```
plt.show()
fig.savefig('./eda_images/rgb_permutations.png')
```



These distributions mostly overlap, unfortunately. There might be a small area where dl1 is distinguishable from dl3 in the red, but it's not really distinct enough to assist in population separation.

6.2 How are the distributions “separable”?

The below are a non-exhaustive list of ways we can think about classifying these disaster images. You do not have to include all of them in your analysis.

6.2.1 1. midwest-flooding vs social-fire

Among the different disaster events, are there are distinctions in:

- image height, width, or overall size?
- RGB values?
- Sobel edges?
- Local Binary Patterns?
- Gabor kernels?

[]:

[]:

6.2.2 2. Damage levels 1 vs 3 within the hurricane-matthew dataset

Is there a relationship between the damage level and:

- image height, width, or overall size?
- RGB values?
- Sobel edges?
- Local Binary Patterns?
- Gabor kernels?

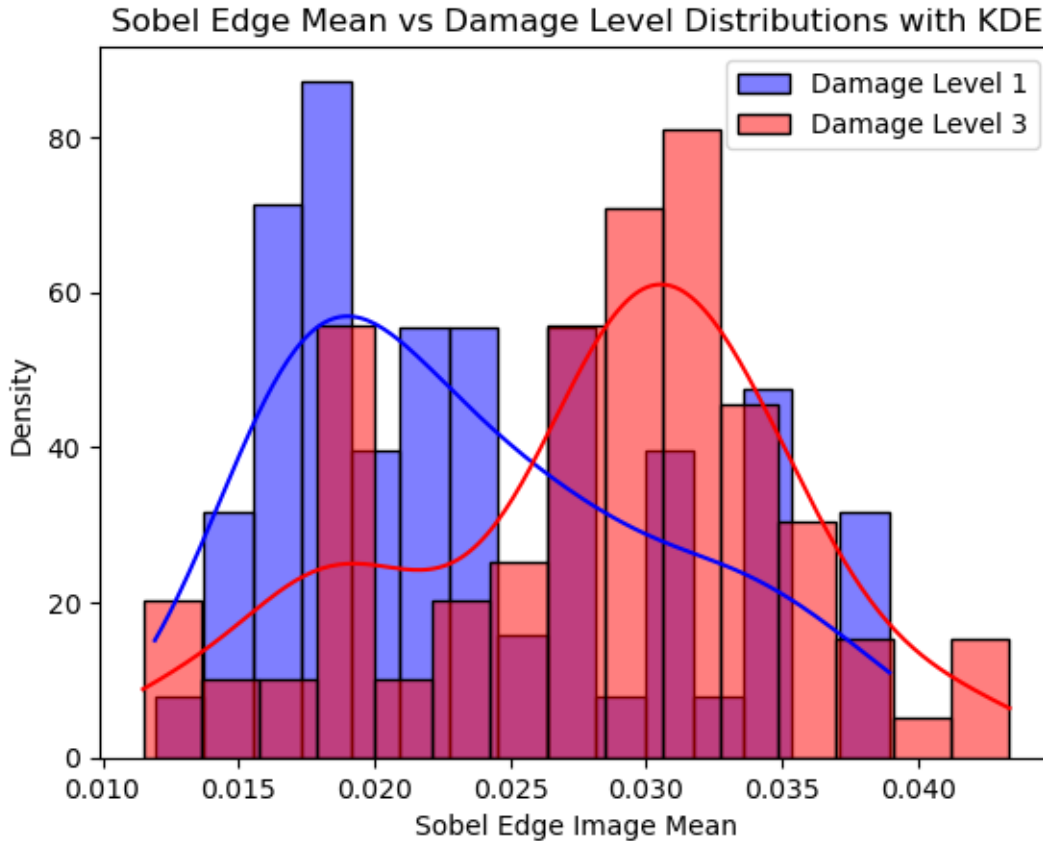
Since as we observed there is no difference in image height, width, or overall size, this is not a good separating factor. As we saw in the color section, not very separable by color either.

So next, let's look at sobel edges, as it might be a good separating factor based on what we saw when we visually inspected the images. For damage level 1 many "lines" were intact, like roads, edges of farmland, etc., whereas for damage level 3 they were not. We get the sobel feature detection's "edge" images to quantify these lines, and take the overall mean and standard deviation of the sobel edge images.

```
[23]: # Get stats from the sobel images
def sobel_stats(images):
    features = []
    for img in images:
        sobel_edges = get_sobel_features(img)
        features.append([np.mean(sobel_edges), np.std(sobel_edges)]),
    return np.array(features)

# Pull out stats from damage level 1 and 3 images
edges_dl1 = sobel_stats(himgs_dl1)
edges_dl3 = sobel_stats(himgs_dl3)

[24]: # Plot sobel edge means as a normalized histogram distribution for each ↵
      ↵population
fig = plt.figure()
sns.histplot(edges_dl1[:,0], bins=15, kde=True, stat='density', color='blue', ↵
      ↵label='Damage Level 1', alpha=0.5)
sns.histplot(edges_dl3[:,0], bins=15, kde=True, stat='density', color='red', ↵
      ↵label='Damage Level 3', alpha=0.5)
plt.xlabel('Sobel Edge Image Mean')
plt.ylabel('Density')
plt.title('Sobel Edge Mean vs Damage Level Distributions with KDE')
plt.legend()
plt.show()
fig.savefig('./eda_images/sobel_mean.png')
```



We find that the sobel edge mean is actually a decent feature to use to separate the damage level 1 and 3 populations. The standard deviation was similar between the classes.

Next, let's investigate something else observed from visual inspection, which is that there is more structure in lines for lower damage levels, i.e. they were laid out in grid patterns and/or were linear. So we want to quantify how linear / structured the sobel edges are, which we achieve via the sobel edge coherence.

```
[25]: from scipy import ndimage
from PIL import Image

# Computes the coherence (essentially calculates linearity / parallelity of
# ↪ sobel gradient)
def sobel_coherence(image):
    # Turn the image to grayscale, as only intensity differences matter
    img = np.asarray(Image.fromarray(image, "RGB").convert('L')) / 255.0
    # Sobel edge detection in the x direction
    gx = ndimage.sobel(img, axis=0)
    # Sobel edge detection in the y direction
    gy = ndimage.sobel(img, axis=1)
    # Get the magnitude of the gradient
```

```

mag = np.hypot(gx, gy)
# Get the angle of the gradient
theta = np.arctan2(gy, gx)

# Weighted directional coherence (range 0-1)
c = np.sum(mag * np.cos(2*theta))
s = np.sum(mag * np.sin(2*theta))

# Coherence
coherence = np.sqrt(c**2 + s**2) / np.sum(mag)
return coherence

# Compute per-image coherence for each class
coh_dl1 = [sobel_coherence(f) for f in himgs_dl1]
coh_dl3 = [sobel_coherence(f) for f in himgs_dl3]

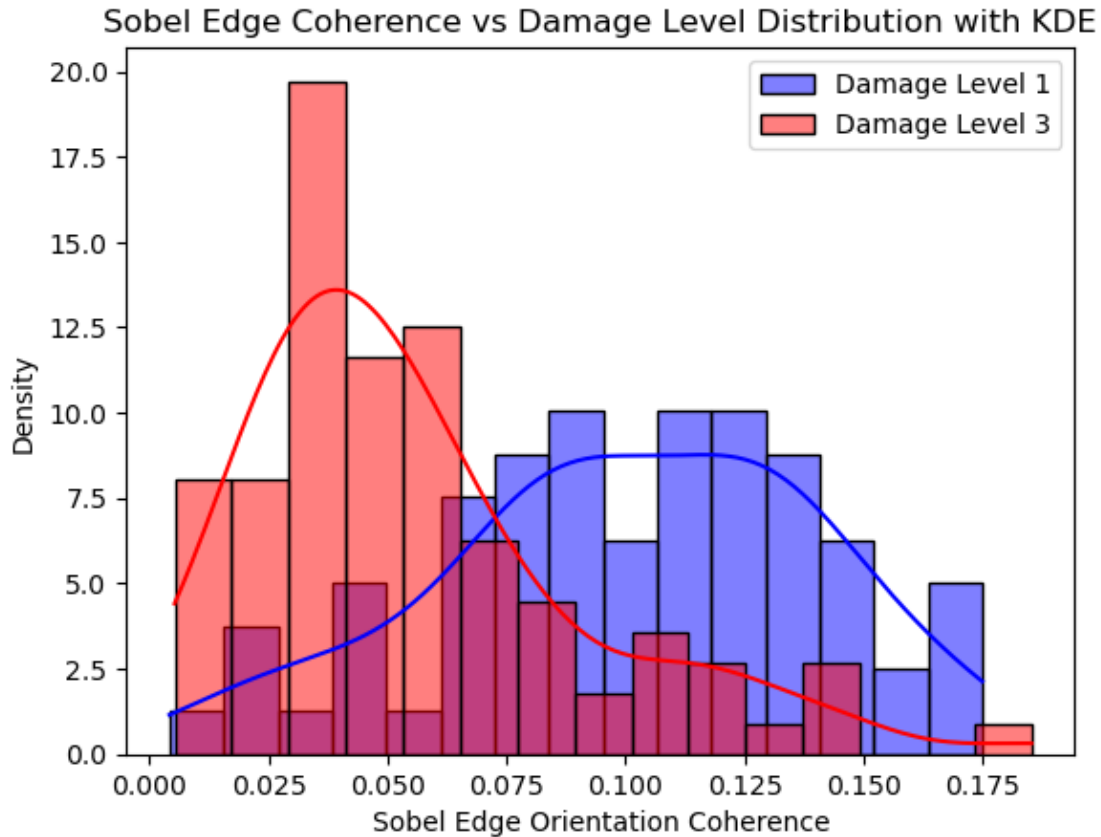
```

```

[26]: # Histogram comparing the distributions of coherence for each damage level
      ↪population
fig = plt.figure()
sns.histplot(coh_dl1, bins=15, kde=True, stat='density', color='blue',
      ↪label='Damage Level 1', alpha=0.5)
sns.histplot(coh_dl3, bins=15, kde=True, stat='density', color='red',
      ↪label='Damage Level 3', alpha=0.5)
plt.xlabel('Sobel Edge Orientation Coherence')
plt.ylabel('Density')
plt.title('Sobel Edge Coherence vs Damage Level Distribution with KDE')
plt.legend()
plt.show()

fig.tight_layout()
fig.savefig('./eda_images/sobel_edge_coherence.png')

```



As we observe, this is another feature that does a decent job of separating the two classes. This makes sense, as it essentially suggests the less damaged areas have more order to them.

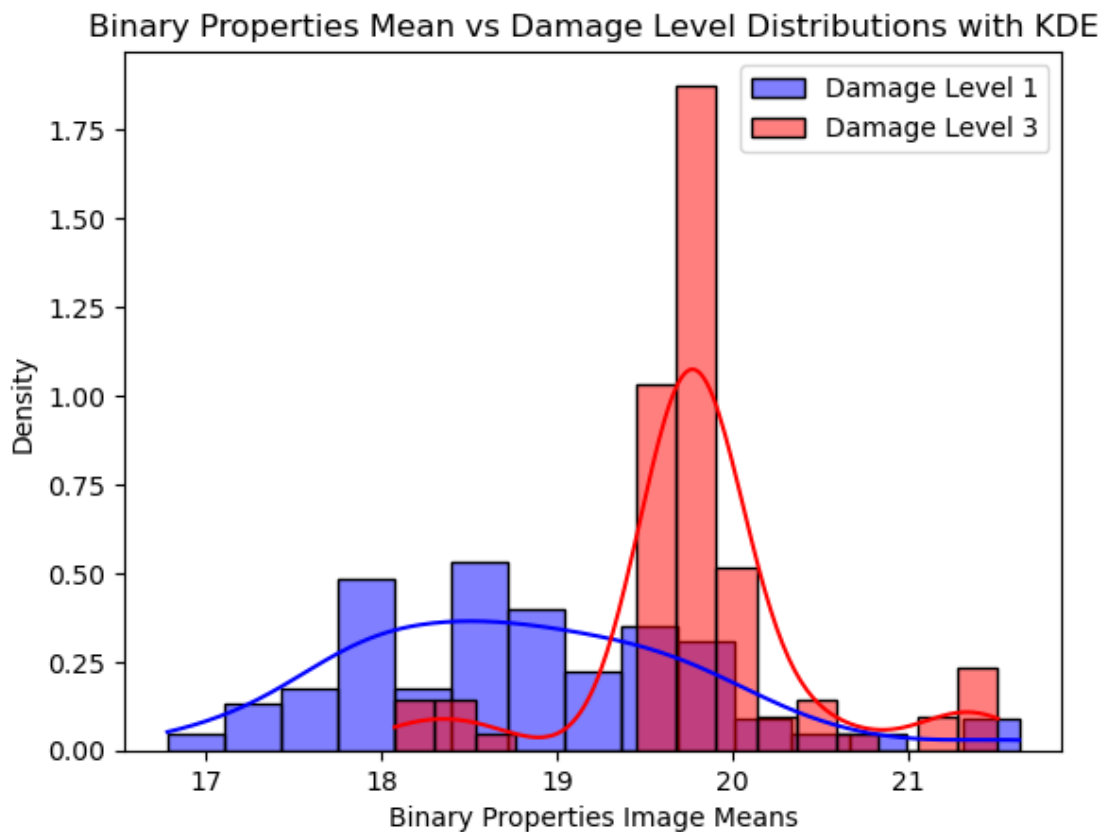
Now let's look at the binary patterns in the images. This describes the local texture of each pixel of the image. In theory for higher damage images, we may see that there is overall higher amounts of texture due to the damage. We get the means of all of the local binary images for each class to assess this.

```
[27]: # Get stats from the binary images
def binary_stats(images, radius=3):
    features = []
    for img in images:
        local_binary = get_local_binary_pattern(img, radius=radius)
        features.append([np.mean(local_binary), np.std(local_binary)]),
    return np.array(features)

# Calculate binary property stats for damage levels 1 and 3
binary_props_dl1 = binary_stats(himgs_dl1)
binary_props_dl3 = binary_stats(himgs_dl3)
```

```
[28]: # Plot binary property means as a histogram distribution for each population
fig = plt.figure()
sns.histplot(binary_props_dl1[:,0], bins=15, kde=True, stat='density',
             color='blue', label='Damage Level 1', alpha=0.5)
sns.histplot(binary_props_dl3[:,0], bins=15, kde=True, stat='density',
             color='red', label='Damage Level 3', alpha=0.5)
plt.xlabel('Binary Properties Image Means')
plt.ylabel('Density')
plt.title('Binary Properties Mean vs Damage Level Distributions with KDE')
plt.legend()
plt.show()

fig.tight_layout()
fig.savefig('./eda_images/binary_props.png')
```

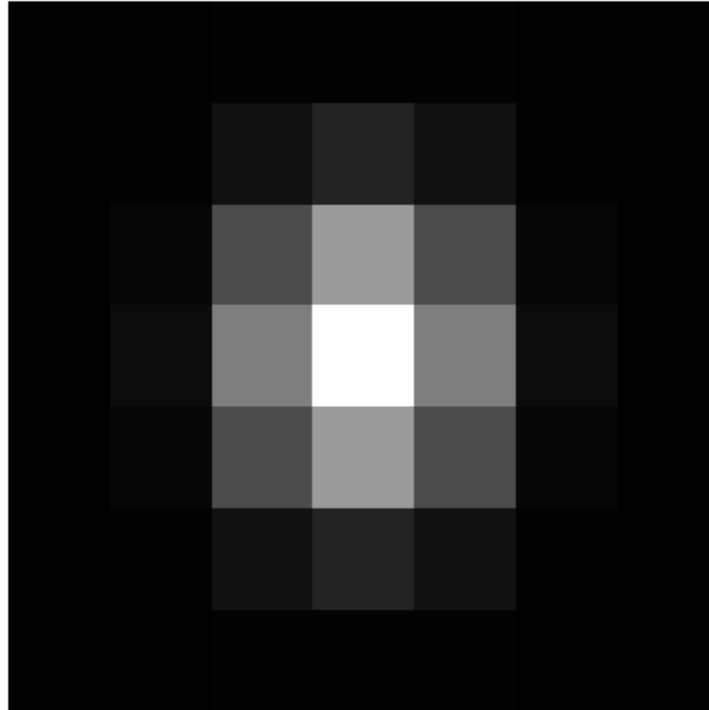


This is also a feature which results in a decent separation between the two populations.

As a final investigation, we want to look at gabor kernels. We try the plus shaped kernel, as it might be an indicator of perpendicular lines / other infrastructure type structures, which will be more prevalent in the less damaged set of images.


```
[29]: # We use the following kernel, which is plus shaped
theta = 0
sigma = 1.0
frequency = 0.1

plus_kernel = generate_gabor_kernel(theta, sigma, frequency)
fig, ax = plt.subplots()
ax.imshow(plus_kernel, cmap="gray")
ax.axis("off");
```



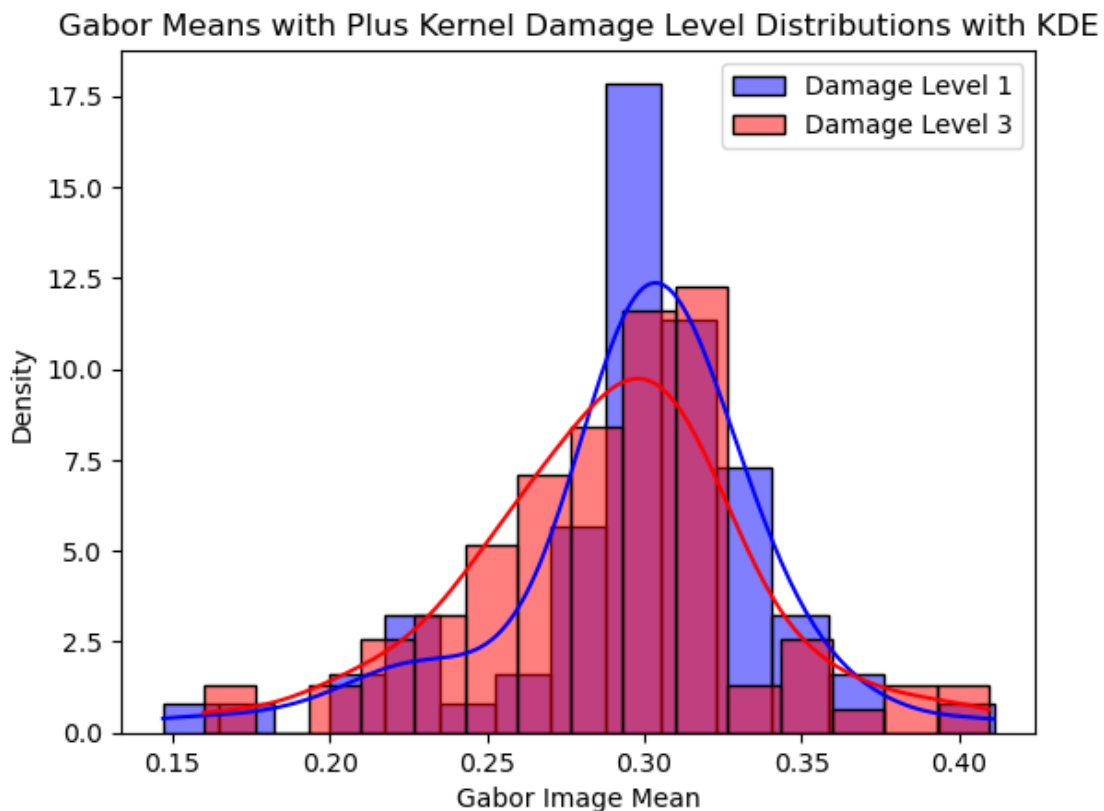
We convolve every image by this gabor kernel, and then take the mean to see if one or another damage level contains more of this pattern.

```
[30]: # Get stats from the binary images
def gabor_stats(images, kernel):
    features = []
    for img in images:
        gabor = get_gabor_features(img, kernel)
        features.append([np.mean(gabor), np.std(gabor)]),
    return np.array(features)

# Get gabor stats for damage level 1 and 3
gabor_props_dl1 = gabor_stats(himgs_dl1, plus_kernel)
```

```
gabor_props_dl3 = gabor_stats(himgs_dl3, plus_kernel)
```

```
[31]: # Plot gabor means as a normalized histogram distribution for each population
sns.histplot(gabor_props_dl1[:,0], bins=15, kde=True, stat='density',
             color='blue', label='Damage Level 1', alpha=0.5)
sns.histplot(gabor_props_dl3[:,0], bins=15, kde=True, stat='density',
             color='red', label='Damage Level 3', alpha=0.5)
plt.xlabel('Gabor Image Mean')
plt.ylabel('Density')
plt.title('Gabor Means with Plus Kernel Damage Level Distributions with KDE')
plt.legend()
plt.show()
```



Unfortunately, this feature does not result in meaningful separation of the two populations. It may be that this only accounts for one orientation of the kernel, and there is no guarantee that perpendicularities in the image are aligned with the x and y axes.

In summary, we observed three parameters that result in meaningful class separation, which are binary property means, sobel edge coherence, and sobel edge means. We want to investigate these features to see if they are correlated with each other, to ensure we wouldn't be using redundant features. We combine the classes here, as correlation between the features shouldn't be class dependent.

```
[32]: # Concatenate the binary props, using first dimension (mean)
binary_props_dls = np.concatenate((binary_props_dl1[:,0], binary_props_dl3[
    ↪,0]))

# Concatenate the coherences
coh_dls = np.concatenate((coh_dl1, coh_dl3))

# Concatenate the sobel edge means, using first dimension (mean)
edges_dls = np.concatenate((edges_dl1[:,0], edges_dl3[:,0]))

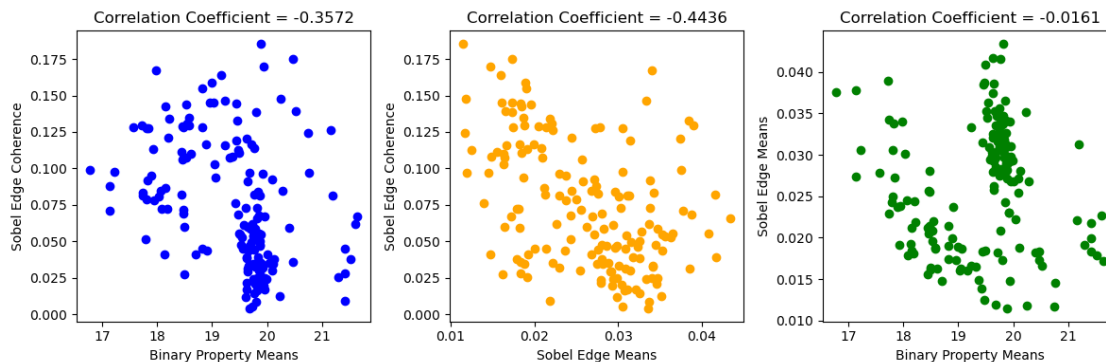
[33]: # Scatter plot the permutations of the three separable parameters
fig, ax = plt.subplots(1, 3, figsize=(12, 4))

r_0 = np.corrcoef(binary_props_dls, coh_dls)[0][1]
ax[0].scatter(binary_props_dls, coh_dls, color='blue')
ax[0].set_xlabel('Binary Property Means')
ax[0].set_ylabel('Sobel Edge Coherence')
ax[0].set_title('Correlation Coefficient = ' + str(np.round(r_0, decimals=4)))

r_1 = np.corrcoef(edges_dls, coh_dls)[0][1]
ax[1].scatter(edges_dls, coh_dls, color='orange')
ax[1].set_xlabel('Sobel Edge Means')
ax[1].set_ylabel('Sobel Edge Coherence')
ax[1].set_title('Correlation Coefficient = ' + str(np.round(r_1, decimals=4)))

r_2 = np.corrcoef(binary_props_dls, edges_dls)[0][1]
ax[2].scatter(binary_props_dls, edges_dls, color='green')
ax[2].set_xlabel('Binary Property Means')
ax[2].set_ylabel('Sobel Edge Means')
ax[2].set_title('Correlation Coefficient = ' + str(np.round(r_2, decimals=4)))

fig.tight_layout()
fig.savefig('./eda_images/feature_correlations.png')
```



We observe very little correlation in the different parameters, meaning collectively they make a useful, and non-redundant, set of features to use to separate damage level 1 from damage level 3 images.

[]: